

container-images 개요

`container-images` 는 GPU container가 시작된 뒤 사용자가 바로 작업할 수 있도록 계정, 공유 홈, SSH, Jupyter, VNC와 Kerberos 환경을 구성하고 CUDA/TensorFlow 조합별 image를 제공한다. **설계**는 공통 entrypoint의 런타임 구성과 이미지 버전을 설명하고, **운영**은 빌드·배포에 사용하는 명령, 테스트 전략과 변경 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	개요, 계정·홈·SSH·Jupyter·VNC·Kerberos 구성, CUDA/TensorFlow 조합별 이미지, 디렉터리 지도
운영	빌드/배포 흐름, 테스트 전략, 변경 가이드, 운영 안전 수칙

처음 보는 사람은 **설계 문서**에서 이미지가 무엇을 책임지고 무엇을 책임지지 않는지 먼저 확인하고, 실제로 빌드하거나 새 이미지 버전을 추가해야 할 때 **운영 문서**로 이동한다.

container-images 설계

개요 · 운영

GitHub 코드 링크: `admin_infra_server` 는 비공개 저장소다. 코드 링크를 누르면 GitHub 로그인 화면을 거쳐 원래 파일과 line으로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

`container-images` 의 목표는 GPU container가 시작된 직후 사용자가 자신의 계정과 공유 홈으로 접속하고, 선택한 GPU software 환경에서 바로 작업을 시작할 수 있게 하는 것이다.

`container-images` 는 GPU container용 image(`Dockerfile` 과 `image-variants.json`)와, 그 image에 담겨 있다가 container가 시작될 때 자동 실행되는 `entrypoint.sh` 를 관리하는 모듈이다. 실제로 container를 만들고 실행하는 행위(`docker run`)는 `infra` 라는 별도 시스템이 수행하며, `container-images` 의 역할은 그 실행이 시작된 뒤 image에 포함된 `entrypoint.sh` 가 넘겨받은 입력으로 사용자 환경을 구성하는 지점부터다.

모든 container는 전달받은 UID/GID로 계정과 그룹을 구성한다. 홈 디렉터리는 컨테이너 밖의 공유 스토리지이므로, 그 안에 이미 있는 `.bashrc` 같은 사용자 설정 파일이나 `vnc_password.txt` 같은 서비스별 password 파일이 있으면 새로 만들지 않고 그대로 이어서 사용한다. SSH와 Jupyter를 기본으로 제공하고, 필요한 경우 VNC와 Kerberos ccache

환경도 함께 구성한다. UID/GID와 홈이 항상 같은 값으로 전달되므로, container를 재시작하거나 다른 이미지 버전을 선택해도 같은 사용자(identity)와 작업 환경을 유지한다.

계정, supplemental group, 홈 디렉터리와 Kerberos 정보처럼 사용자와 host마다 달라지는 값은 공통 `entrypoint.sh`가 시작 시 반영한다. CUDA, TensorFlow, Python과 Ubuntu 조합은 각각의 이미지 버전으로 제공한다. 따라서 사용자는 필요한 GPU software 조합을 선택하면서도 모든 이미지 버전에서 동일한 방식으로 SSH, Jupyter, VNC와 공유 홈을 사용할 수 있다.

2. 사용자 실행 환경

`Dockerfile`은 모든 이미지 버전에 필요한 프로그램과 `entrypoint.sh`를 image에 포함한다. 실제 사용자 정보와 실행 옵션은 container를 생성하는 외부 시스템이 전달하며, `entrypoint`가 이를 검증하고 런타임 환경을 구성한다.

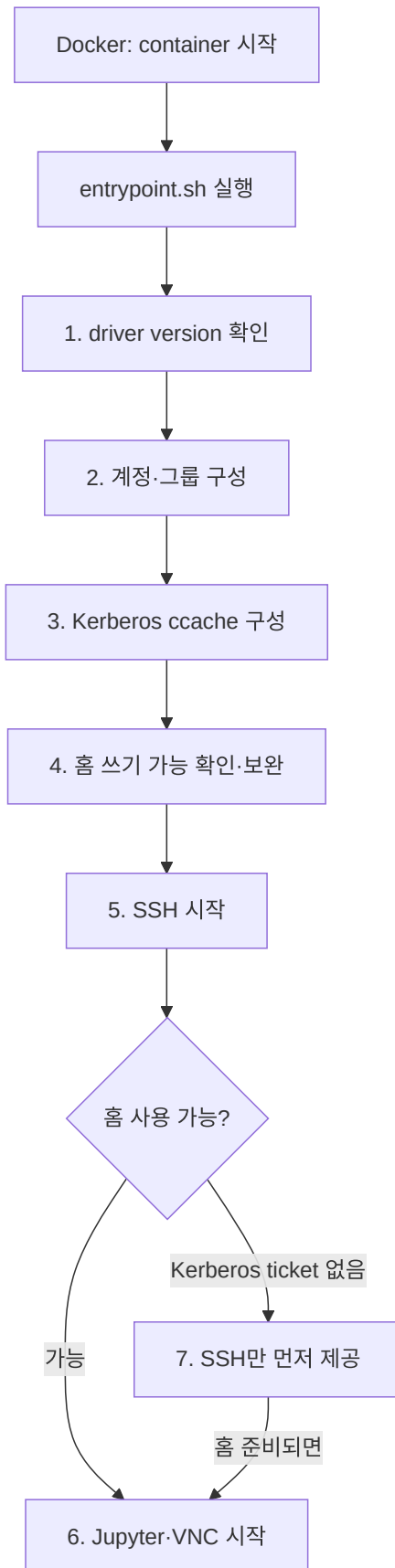
`container-images`는 사용자 identity, group membership, host mount, port와 Kerberos ticket을 결정하지 않는다. 즉 image에는 특정 사용자의 계정이나 홈을 미리 넣지 않는다 — 이 값들은 container 생성 전에 외부 시스템이 정해 전달하는 runtime 입력이다. 이 문서에서는 그 입력이 만들어지는 과정이 아니라, image와 `entrypoint.sh`가 입력을 받아 container 내부 환경을 구성하는 동작을 설명한다.

구분	이 문서에서 다루는 범위
외부에서 전달되는 입력	UID/GID와 그룹 목록, <code>/home</code> mount, Kerberos ccache, port mapping과 실행 option
<code>container-images</code> 가 수행하는 작업	image에 필요한 프로그램을 포함하고, 전달된 입력을 container 내부 계정·파일·service 설정에 반영한다.

같은 image를 사용하더라도, 외부 시스템이 container를 생성할 때 넘겨주는 identity, mount와 실행 옵션에 따라 `entrypoint`가 각 사용자의 환경을 구성한다.

2.1 컨테이너 시작 흐름

Docker가 `entrypoint.sh`를 container의 시작 프로세스로 실행한다(image에 Docker `ENTRYPOINT`로 지정되어 있다). 아래 7단계는 전부 이 하나의 스크립트 안에서 순서대로 호출되는 함수이며, 별도의 `.py`나 다른 프로세스가 아니다. 다음 순서는 `container-images` 내부에서 수행하는 작업만 나타낸다.



1. image에 기록된 CUDA/TensorFlow version을 출력하고 host NVIDIA driver가 최소 version을 충족하는지 확인한다.
2. 전달받은 UID/GID로 사용자, primary group과 supplemental group을 구성한다.
3. 전달받은 Kerberos ccache를 shell과 사용자 process에서 사용할 수 있게 구성한다.

4. mount되어 있는 홈 디렉터리의 쓰기 가능 여부를 사용자 권한으로 확인하고 초기 설정을 보완한다.
5. SSH 로그인, audit와 login message를 구성하고 SSH service를 시작한다.
6. 홈을 사용할 수 있으면 Jupyter와 선택적인 VNC/noVNC를 시작한다.
7. Kerberos ticket이 없어 홈을 쓸 수 없다면 SSH만 먼저 제공하고, 홈이 준비된 뒤 사용자 서비스를 시작한다.

이 순서를 하나의 entrypoint에서 관리하므로 이미지 버전마다 사용자 환경을 별도로 구현하지 않는다.

관련 코드

- **entrypoint의 전체 실행 순서**: 계정, Kerberos, 홈, SSH와 사용자 service를 순서대로 호출한다.
- **start_user_apps**: 사용자 shell 환경을 보완한 뒤 Jupyter와 VNC/noVNC를 시작한다.

2.2 계정, 그룹과 권한

사용자 계정과 group membership은 외부에서 결정되고, 공유 스토리지는 그 결과인 UID/GID를 기준으로 파일 권한을 판단한다. container 안에서 실행되는 process도 같은 권한을 사용하려면 Linux 사용자와 그룹이 동일한 UID/GID를 가져야 한다.

따라서 entrypoint는 전달된 값으로 container 내부 사용자와 primary group을 만들고, 전달된 supplemental group만 같은 GID로 추가한다. 사용자가 어떤 그룹에 속해야 하는지는 판단하지 않는다. 같은 이름의 사용자나 그룹이 다른 UID/GID로 이미 존재하면 외부 권한과 다르게 동작할 수 있으므로 container 시작을 중단한다.

계정과 그룹을 만드는 것 외에, entrypoint는 사용자가 나중에 직접 쓸 수 있는 그룹 공유 도구도 하나 설치해 둔다. Kerberos 공유 스토리지에서는 사용자가 자신의 홈 directory를 배정된 그룹과 공유할 수 있도록 **group-dir-share** 명령을 제공한다. **Dockerfile**은 ACL 도구를 포함하고, entrypoint는 Kerberos 환경을 구성할 때 이 명령을 **/usr/local/bin**에 생성해 둔다. 이 명령을 실행하는 주체는 entrypoint가 아니라 로그인한 사용자다.

```
group-dir-share ~/project vision
```

이 명령은 사용자가 해당 그룹에 속하는지와 경로가 사용자 홈 안에 있는지를 확인한 뒤 group owner, **2770** mode와 ACL을 설정한다. 별도의 권한을 부여하는 명령이 아니라 실행한 사용자가 이미 가진 group membership 범위에서만 동작한다.

계정·그룹 구성과 별개로, entrypoint는 sudo 권한도 함께 제어한다. 기본 sudo mode인 **restricted**는 package 설치에 필요한 명령은 허용하지만 사용자 전환, mount, 권한 변경, root shell과 우회 가능한 interpreter 실행은 막는다. 기존 사용자의 password는 재시작할 때 변경하지 않으며, **USER_PW**는 사용자를 처음 생성할 때만 적용한다.

entrypoint가 입력으로 받는 주요 값은 다음과 같다.

값	의미
USER_ID	container에 생성하거나 확인할 사용자 이름
UID , GID	DB·AD·NFS와 일치하는 숫자 identity
USER_GROUP	primary group 이름
DECS_SUPPLEMENTAL_GROUPS	추가할 group:GID 목록

값	의미
DECS_USER_SUDO_MODE	disabled, restricted, allowed 중 하나

TARGET_UID 나 TARGET_GID 같은 두 번째 identity 체계를 만들지 않는다. 하나의 UID / GID 만 사용해야 container, 공유 홈과 DB의 소유권이 서로 달라지는 오류를 시작 단계에서 찾을 수 있다.

관련 코드

- ensure_group_and_user : primary group과 사용자를 생성·검증하고 sudo mode를 적용한다.
- ensure_supplemental_groups : 추가 그룹의 이름과 GID가 전달값과 일치하는지 확인한다.
- Dockerfile 의 ACL package 설치: group-dir-share 가 ACL을 구성할 때 사용하는 setfacl 을 image에 포함한다.
- install_kerberos_share_helper : /usr/local/bin/group-dir-share script의 검증과 권한 설정 동작을 정의하고 container 시작 시 파일을 생성한다.
- ensure_kerberos_runtime : Kerberos ccache가 전달된 경우에만 그룹 공유 명령을 생성한다.
- write_restricted_sudoers : 제한적 sudo에서 허용하지 않을 권한 상승 경로를 정의한다.

2.3 홈 디렉터리와 공유 스토리지

공유 스토리지를 host에 mount하고 Docker bind mount로 container에 연결하는 작업은 container 시작 전에 외부에서 수행한다. container-images 는 NFS를 mount하거나 mount source를 선택하지 않는다. entrypoint가 동작할 때는 다음 경로가 이미 container에 연결되어 있다고 가정한다.

전달되는 mount	entrypoint에서 사용하는 위치	사용 목적
사용자 홈의 상위 공유 경로	/home	/home/<USER_ID> 를 사용자 홈으로 사용한다.
사용자 Kerberos ccache directory	host와 같은 경로	사용자 process에 KRB5CCNAME 으로 전달한다.
krb5.conf	/etc/krb5.conf (read-only)	Kerberos client 설정으로 사용한다.

entrypoint가 담당하는 작업은 mount된 /home/<USER_ID> 를 실제 사용자 권한으로 검증하고 container 실행에 필요한 최소 설정을 보완하는 것이다. 홈이 없으면 생성을 시도하고, 전달받은 UID/GID의 사용자로 파일을 직접 써 보아 사용 가능 여부를 확인한다. 공유 홈 전체의 소유권을 변경하지 않으며 .profile, .bashrc, .bash_logout 은 없는 경우에만 추가하며 기존 사용자 파일을 덮어쓰지 않는다.

NFS가 root_squash 를 사용하면 container root도 홈의 owner를 임의로 바꿀 수 없다. 이는 외부 스토리지의 조건이며 entrypoint는 이를 우회하지 않는다. Kerberos ccache가 전달되었지만 아직 홈을 쓸 수 없다면 SSH만 먼저 시작하고, 홈이 쓰기 가능해진 뒤 초기 설정과 Jupyter·VNC 시작을 이어서 수행한다.

관련 코드

- ensure_user_home : 홈 생성, 쓰기 권한, 기본 shell 파일과 history 저장 설정을 처리한다.
- start_kerberos_home_watcher : Kerberos ticket을 기다리면서 홈이 쓰기 가능해지는 시점을 확인한다.

2.4 SSH, Jupyter와 VNC

host port mapping, firewall과 `ENABLE_VNC` 같은 실행 option은 container 시작 전에 외부에서 정한다. `container-images` 는 host port를 할당하거나 network 접근 정책을 설정하지 않고, container 내부 service만 구성한다.

entrypoint는 SSH가 사용자와 관리 계정만 로그인할 수 있도록 `AllowUsers` 를 구성하고, PAM·command audit·login message를 적용한 뒤 SSH service를 재시작한다. 홈이 아직 Kerberos 인증을 기다리는 상태여도 SSH는 먼저 사용할 수 있으므로 사용자가 로그인하여 ticket 상태를 확인할 수 있다.

Jupyter 설정은 사용자 홈의 `.jupyter` 에 둔다. 관리하는 network·notebook 경로 항목만 갱신하고 나머지 사용자 설정은 유지한다. 시작할 때 임의 token을 만들고 권한이 0600인 `jupyter_token.txt` 에 저장한 뒤 사용자 권한으로 JupyterLab을 실행한다.

VNC/noVNC는 `ENABLE_VNC=true` 일 때만 시작한다. TigerVNC는 container 내부의 localhost에 bind하고, noVNC가 지정된 web port를 통해 연결한다. 기존 홈에 `vnc_password.txt` 가 있으면 재사용하므로 container 재시작만으로 password가 바뀌지 않는다.

관련 코드

- `configure_system_login`: SSH 접근 사용자, PAM, audit와 login message를 구성한다.
- `ensure_jupyter_config`: 사용자 설정을 보존하면서 Jupyter의 공통 항목을 갱신한다.
- `start_jupyter`: 접근 token을 저장하고 사용자 권한으로 JupyterLab을 시작한다.
- `start_novnc`: VNC password를 준비하고 TigerVNC와 noVNC proxy를 시작한다.

2.5 Kerberos 사용자 환경

Kerberos principal, ticket 발급·갱신, ccache 생성과 host NFS 인증은 `container-images` 밖에서 준비한다. host의 machine keytab도 container에 전달하지 않는다.

`container-images` 가 담당하는 작업은 전달받은 ccache를 container의 사용자 process에서 사용할 수 있게 만드는 것이다. entrypoint는 `KRB5CCNAME` 이 있을 때 ccache directory의 접근 권한, shell 환경 변수와 `decs-kerberos-status` 명령을 구성하고 Jupyter와 VNC에도 같은 환경 변수를 전달한다. 사용자는 `decs-kerberos-status` 로 ticket 상태를 확인할 수 있지만, entrypoint가 ticket을 직접 발급하거나 갱신하지는 않는다.

Kerberos keytab과 ccache를 분리하는 이유와 credential 경계는 Kerberos/NFS의 keytab과 ccache 모델을 따른다.

관련 코드

- `ensure_kerberos_runtime`: ccache 경로의 권한과 Kerberos 환경 변수, ticket 확인 helper를 구성한다.

3. 이미지 구성

사용자 실행 환경은 모든 image에서 같지만 GPU software stack은 workload에 따라 달라진다. `container-images` 는 공통 image 구성과 CUDA/TensorFlow 조합만 분리해 관리한다.

3.1 모든 이미지 버전의 공통 build 구성

모든 이미지 버전은 하나의 `Dockerfile` 을 사용하고 다음 프로그램을 공통으로 포함한다.

영역	공통 구성
기본 도구	ACL, audit, network, certificate와 package repository 도구
원격 접근	OpenSSH server와 Kerberos client
개발 환경	Miniforge, conda, pip, JupyterLab과 Notebook
GUI	Chrome, Xfce, TigerVNC, noVNC와 websockify
사용자 환경	한국어 글꼴과 입력기, 공통 entrypoint와 Jupyter config 위치

이미지 버전별 Dockerfile을 복제하지 않는 이유는 보안 patch와 공통 package 변경을 한 번만 적용하기 위해서다. 실제 차이는 build argument로 전달하며 공통 설치와 시작 흐름은 모든 이미지 버전에서 동일하게 유지한다.

3.2 CUDA/TensorFlow 조합별 이미지

이미지 버전 간 차이는 `image-variants.json`에 정의한다.

manifest 값	이미지 버전마다 달라지는 내용
<code>base_image</code>	CUDA, cuDNN과 Ubuntu가 포함된 NVIDIA base image
<code>cuda_version</code>	image가 제공하는 CUDA major/minor version
<code>tensorflow_version</code> , <code>tensorflow_package</code>	CUDA 조합에 맞춰 설치할 TensorFlow version과 package 형태
<code>python_version</code> , <code>conda_packages</code>	Python version과 TensorFlow/Jupyter 호환을 위한 package 제약
<code>min_nvidia_driver</code>	해당 CUDA image를 실행할 host의 최소 NVIDIA driver
<code>support</code>	실장비 검증을 마친 <code>stable</code> 또는 검증 중인 <code>experimental</code> 상태
<code>aliases</code>	version alias와 <code>stable</code> , <code>latest</code> 같은 권장 tag

현재 Python은 모두 3.10, Ubuntu는 모두 22.04를 사용한다. CUDA 조합에 따라 TensorFlow package, conda 제약과 최소 host driver가 달라진다.

CUDA	TensorFlow	주요 package 차이	최소 NVIDIA driver	지원 상태와 alias
11.8	2.13.0	구버전 Jupyter·IPython과 <code>typing_extensions=4.5</code> 제약	520.61.05	<code>stable</code> , <code>legacy</code>
12.2	2.15.0	기본 TensorFlow package와 공통 Jupyter package	535.104.05	<code>stable</code>
12.3	2.16.1	<code>tensorflow[and-cuda]</code> package 사용	545.23.08	<code>stable</code>
12.5	2.20.0	TensorFlow 2.20 기준 이미지	555.42.06	<code>stable</code> , <code>latest</code>

CUDA	TensorFlow	주요 package 차이	최소 NVIDIA driver	지원 상태와 alias
12.8	2.20.0	H200 대상 실험 이미지	570.124.06	experimental, h200-experimental

로컬 빌드 도구와 GitHub Actions는 같은 manifest를 읽어 build argument와 tag를 만든다. 따라서 지원 조합을 한 곳에서 검토하고 로컬과 CI가 서로 다른 image를 생성하는 것을 막는다.

3.3 GPU 호환성 처리

image는 시작할 때 build에 기록된 최소 NVIDIA driver와 `nvidia-smi` 결과를 출력한다. 기본값은 경고 중심이며 `STRICT_CUDA_COMPAT=true` 일 때만 최소 driver보다 낮은 host에서 시작을 실패시킨다.

기본을 강제 실패로 두지 않은 이유는 GPU가 없는 검증 환경이나 긴급 진단 container까지 막지 않기 위해서다. 운영 배포에서 호환성을 반드시 보장해야 하면 strict mode를 명시한다.

관련 코드

- `Dockerfile`: 공통 package를 설치하고 manifest 값을 build argument로 받는다.
- `image-variants.json`: 지원하는 이미지 버전별 CUDA/TensorFlow 조합, 최소 driver와 tag를 정의한다.
- `print_image_runtime_info`: image의 요구 driver와 실제 host driver를 비교한다.

4. 디렉터리 지도

경로	핵심 기능
<code>Dockerfile</code>	모든 이미지 버전이 공유하는 단일 image 정의
<code>image-variants.json</code>	base image, CUDA/TensorFlow, 최소 driver와 alias 목록
<code>entrypoint.sh</code>	시작 시 계정·그룹·홈·SSH·Jupyter·VNC·Kerberos 환경 구성
<code>scripts/variant_matrix.py</code>	manifest를 Docker/GitHub Actions build matrix로 변환
<code>scripts/build_variants.py</code>	로컬 build/push 명령 생성과 실행
<code>scripts/test_image_variants.py</code>	image metadata, TensorFlow와 선택적 GPU smoke test
<code>scripts/test_uid_create_container.py</code>	infra가 전달하는 container 생성 입력값의 dry-run 계약 검증
<code>tests/</code>	root_squash, Kerberos, sudo와 build 설정 회귀 test
<code>.github/workflows/docker-publish.yml</code>	PR merge 또는 수동 실행 시 matrix build/push

container-images 운영

개요 · 설계

GitHub 코드 링크: `admin_infra_server` 는 비공개 저장소다. 코드 링크를 누르면 GitHub 로그인 화면을 거쳐 원래 파일과 line으로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

이 문서의 목표는 `container-images` 설계에서 설명한 image와 `entrypoint.sh` 를 실제로 변경·검증하여 Docker registry에 배포한 뒤, 운영 container에서 사용할 수 있게 만드는 전체 절차를 제공하는 것이다. 새 CUDA/TensorFlow 조합을 추가하는 경우뿐 아니라 공통 package, 사용자 실행 환경, tag 정책이나 build 도구를 변경할 때 확인해야 할 범위도 함께 다룬다.

하나의 변경은 다음 순서로 운영한다.

1. 변경 목적에 맞는 파일을 선택한다.
2. manifest(`image-variants.json`)와 shell 회귀 test(`tests/*.sh` , 이번 변경이 기존 동작을 깨지 않았는지 image build 없이 먼저 확인하는 test)로 build 입력과 runtime 계약(entrypoint가 특정 입력에 대해 정해진 대로 동작하는지)을 확인한다.
3. 새로운 날짜 tag로 image를 build한다.
4. CPU smoke test(핵심 기능만 빠르게 도는지 확인하는 test)와 대상 GPU host test를 수행한다.
5. 검증한 image를 registry에 push한다.
6. 날짜 tag를 지정하여 신규 container를 생성하거나 기존 container를 교체한다.

`stable` 과 `latest` 는 고정된 release가 아니라 다른 image를 가리킬 수 있는 alias다. build, 배포, 장애 분석 기록에는 `cuda12.5-tf2.20-ubuntu22.04-260706` 처럼 날짜가 포함된 전체 tag를 사용한다.

2. 목적별 수정 위치

변경하려는 목적을 먼저 정한 뒤 그 목적을 소유하는 파일을 수정한다. 하나의 변경이 image 구성, 시작 동작과 검증에 함께 영향을 주면 표의 관련 파일도 같이 변경한다.

변경 목적	주로 수정할 위치	함께 확인하거나 수정할 위치
새로운 CUDA·TensorFlow·Python·Ubuntu 조합을 제공한다.	<code>image-variants.json</code>	<code>README.md</code> 의 image·alias 목록, matrix 출력과 GPU smoke test
모든 이미지 버전에 package나 공통 파일을 추가·갱신한다.	<code>Dockerfile</code>	<code>test_image_build_config.sh</code> , 전체 이미지 버전 build

변경 목적	주로 수정할 위치	함께 확인하거나 수정할 위치
container 시작 시 계정·그룹·홈·Kerberos·SSH·Jupyter·VNC를 구성하는 동작을 바꾼다.	<code>entrypoint.sh</code>	<code>test_entrypoint_root_squash.sh</code> , 기존 홈과 재시작 동작
image repository, 날짜 tag, alias 또는 지원 상태를 바꾼다.	<code>image-variants.json</code> , <code>variant_matrix.py</code>	<code>build_variants.py</code> 의 dry-run tag
로컬 build 명령이나 build argument 전달 방식을 바꾼다.	<code>build_variants.py</code>	<code>variant_matrix.py</code> , <code>Dockerfile</code> 의 ARG, 배포 workflow의 build argument
image에서 확인할 package·TensorFlow·GPU 항목을 추가한다.	<code>test_image_variants.py</code>	CPU test와 대상 GPU host test 결과
Docker Hub build/push를 자동화하거나 trigger를 변경한다.	<code>docker-publish.yml</code>	workflow 위치, build context, Docker Hub secret과 release tag

특정 사용자의 UID/GID, group membership, 홈 mount나 port를 변경하는 것은 `container-images` 변경 목적이 아니다. 외부에서 전달된 값을 container 내부에서 처리하는 방식 자체를 바꿀 때만 `entrypoint.sh`를 수정한다.

3. 새 이미지 버전 추가

3.1 조합 결정

새 이미지 버전은 CUDA 숫자만 추가하는 작업이 아니다. 다음 항목을 하나의 호환 조합으로 결정해야 한다.

- NVIDIA base image의 CUDA, cuDNN과 Ubuntu version
- TensorFlow version과 설치 package 표현
- Python version과 conda package 제약
- 해당 CUDA를 실행할 host의 최소 NVIDIA driver
- 실장비 검증 여부를 나타내는 `support`
- 날짜 tag 외에 제공할 alias

검증을 시작하는 단계에서는 `support`를 `experimental`로 두고 `stable`이나 `latest` alias를 부여하지 않는다. CPU와 대상 GPU test를 통과한 뒤에 지원 상태와 권장 alias를 변경한다.

3.2 manifest 항목 추가

`image-variants.json`의 `variants`에 다음 형태의 항목을 추가한다.

```
{
  "id": "cudaX.Y-tfA.B-ubuntu22.04",
  "base_image": "nvidia/cuda:<CUDA-cuDNN-Ubuntu tag>",
  "cuda_version": "X.Y",
  "tensorflow_version": "A.B.C",
  "tensorflow_package": "tensorflow==A.B.C",
  "conda_packages": "ipywidgets jupyterlab micromamba notebook pip",
  "python_version": "3.10",
  "ubuntu_version": "22.04",
  "min_nvidia_driver": "<minimum driver>",
  "support": "experimental",
  "aliases": [
    "cudaX.Y-tfA.B-experimental"
  ]
}
```

`id`에는 날짜를 넣지 않는다. `build_variants.py`가 `default_date_tag` 또는 `--date-tag` 값을 붙여 실제 배포 tag를 만든다. `aliases`에는 날짜 없이 계속 사용할 이름만 둔다. 기존 `stable`이나 `latest`를 새 항목으로 옮기면 다음 push부터 그 alias가 새 image를 가리키므로 별도의 승격 변경으로 검토한다.

정적 목록을 제공하는 `container-images/README.md`의 이미지 표와 alias 표도 함께 갱신한다. 그다음 `variant_matrix.py`가 manifest에서 만들어내는 matrix와 tag를 확인한다.

```
cd /home/jy/server_manage/container-images
python3 scripts/variant_matrix.py \
  --variant cudaX.Y-tfA.B-ubuntu22.04 \
  --date-tag YYMMDD
```

출력의 `base_image`, build argument와 `docker_tags`가 추가한 항목과 일치해야 한다.

4. 기존 구성 변경

4.1 공통 package와 build 환경 변경

apt package, Miniforge·conda, Python package 설치 순서와 image에 포함할 공통 파일은 `Dockerfile`에서 변경한다. 모든 이미지 버전이 하나의 Dockerfile을 공유하므로 이 변경은 특정 CUDA 조합에만 국한되지 않는다.

변경 후에는 최소한 현재 권장 이미지와 package 제약이 가장 강한 legacy 이미지를 각각 build한다. release 전에는 전체 이미지 버전의 build와 smoke test를 확인한다. build 순서나 필수 package 제약이 달라졌다면 `test_image_build_config.sh`도 같이 수정한다.

4.2 사용자 실행 환경 변경

계정과 그룹 생성, 홈 초기화, Kerberos credential, SSH, Jupyter와 VNC 동작은 `entrypoint.sh` 에서 변경한다. 다음 조건을 유지해야 한다.

- 같은 입력으로 container를 재시작해도 계정과 설정이 중복되거나 손상되지 않는다.
- 기존 공유 홈의 `.bashrc`, Jupyter 설정과 password 파일을 임의로 덮어쓰지 않는다.
- NFS `root_squash` 환경에서 공유 홈 전체를 `chown` 하지 않는다.
- 홈과 사용자 application에 쓰는 파일은 실제 사용자 UID/GID로 생성한다.
- Kerberos keytab을 container에 전달하지 않고 ccache만 사용한다.
- restricted sudo의 identity 전환, mount와 권한 변경 차단을 유지한다.

`entrypoint.sh` 는 image 안에 복사되므로 source만 수정하거나 실행 중인 container를 재시작해도 변경 내용이 적용되지 않는다. 새 날짜 tag로 image를 다시 build·push하고, 운영 container도 그 tag로 다시 생성해야 한다.

4.3 tag, alias와 지원 상태 변경

지원 상태와 alias는 `image-variants.json` 에서 변경한다. `experimental` 을 `stable` 로 승격할 때는 해당 날짜 image의 CPU·GPU smoke 결과와 대상 server의 최소 driver 충족 여부를 먼저 확인한다. `stable` 과 `latest` 는 한 이미지 버전에만 두어 운영자가 어떤 조합이 권장 버전인지 판단할 수 있게 한다.

이미 배포 기록에 사용한 날짜 tag는 같은 이름으로 다시 build하지 않는다. source가 달라졌다면 새로운 `YYMMDD` tag를 사용한다. 같은 tag를 덮어쓰면 target host에 남아 있는 image cache와 registry image가 서로 달라질 수 있다.

4.4 build와 test 도구 변경

manifest field를 추가하거나 tag 규칙을 바꾸면 `variant_matrix.py`, `build_variants.py`, `test_image_variants.py` 가 같은 field와 tag를 사용하는지 함께 확인한다. 로컬 build와 배포 workflow가 서로 다른 image를 만들지 않도록 한쪽에만 build argument를 추가하지 않는다.

5. 로컬 build

모든 명령은 `container-images` directory에서 실행한다.

```
cd /home/jy/server_manage/container-images
```

먼저 실제 build 없이 Docker 명령과 생성될 tag를 확인한다.

```
python3 scripts/build_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD \  
  --dry-run
```

확인한 이미지 버전을 build한다.

```
python3 scripts/build_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD
```

`--variant` 를 생략하면 manifest의 모든 이미지 버전을 순서대로 build한다. Docker cache 때문에 package 갱신이 반영되지 않았는지 확인해야 할 때만 `--no-cache` 를 추가한다.

6. 검증

6.1 source 회귀 test

```
bash tests/test_image_build_config.sh  
bash tests/test_entrypoint_root_squash.sh
```

첫 번째 test는 Dockerfile의 version pin, 필수 package와 설치 순서를 확인한다. 두 번째 test는 entrypoint 문법과 UID/GID, 그룹, sudo, NFS 홈, Kerberos, Jupyter와 VNC의 runtime 계약을 확인한다.

6.2 image smoke test

build한 날짜와 동일한 tag를 지정해 CPU 환경을 확인한다.

```
python3 scripts/test_image_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD
```

이 test는 image에 기록된 TensorFlow version, Python, Jupyter, micromamba와 entrypoint 포함 여부를 확인한다. 실제 NVIDIA runtime과 TensorFlow GPU 인식은 대상 GPU host에서 `--gpu` 를 추가하여 확인한다.

```
python3 scripts/test_image_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD \  
  --gpu
```

6.3 container 생성 계약 확인

image tag가 `infra` 의 container 생성 입력으로 올바르게 전달되는지 실제 DB나 container를 변경하지 않고 확인한다.

```
python3 scripts/test_uid_create_container.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD \  
  --print-only
```

출력된 명령에서 repository 이름과 날짜가 포함된 version tag를 확인한다.

7. registry 배포

7.1 로컬 build와 push

Docker Hub 로그인에 사용할 관리자 계정은 [관리자 계정 통합관리문서](#)에서 확인한다. 인증과 test가 완료된 환경에서 `--push` 를 추가한다.

```
docker login  
python3 scripts/build_variants.py \  
  --variant cuda12.5-tf2.20-ubuntu22.04 \  
  --date-tag YYMMDD \  
  --push
```

이 명령은 날짜 tag와 해당 이미지 버전에 선언된 alias를 모두 push한다. 따라서 `stable` 이나 `latest` 가 포함된 항목은 alias 변경까지 승인된 뒤 실행한다. push 후 대상 GPU host에서 날짜 tag를 pull하고 6.2의 GPU smoke test를 다시 수행한다.

7.2 GitHub Actions 상태

build/push workflow 정의에는 `main` 대상 PR merge와 수동 `workflow_dispatch` 가 trigger로 작성되어 있다. 그러나 현재 파일은 모노레포 최상위 `.github/workflows/` 가 아니라 `container-images/.github/workflows/docker-publish.yml` 에 있으므로 GitHub가 자동 workflow로 등록하지 않는다.

workflow를 최상위 위치로 옮기고 working directory, Docker build context와 `DOCKERHUB_USERNAME · DOCKERHUB_TOKEN` secret을 확인하기 전까지는 PR merge를 image 배포 완료로 간주하지 않는다. 현재 배포 기준은 7.1의 로컬 build/push다.

8. 운영 container 실행과 교체

8.1 신규 container 실행

운영 container는 긴 `docker run` 명령을 직접 작성하지 않고 `infra` 의 생성 명령을 사용한다. 그래야 UID/GID, 그룹 membership, port, GPU, 공유 홈과 Kerberos ccache가 같은 기준으로 준비된다.

먼저 날짜 tag를 넣고 `--dry-run` 으로 계획과 Docker 명령을 확인한다.

```
uidctl create-container \
  --name "사용자 이름" \
  --username username \
  --server-id LAB10 \
  --expiration-date YYYY-MM-DD \
  --image decs \
  --version cuda12.5-tf2.20-ubuntu22.04-YYMMDD \
  --created-by operator \
  --email user@example.com \
  --phone 000-0000-0000 \
  --dry-run
```

계획의 target server, GPU, host port, mount source와 최종 image tag를 검토한 뒤 같은 명령에서 `--dry-run` 만 제거하여 실행한다. Kerberos 또는 VNC가 필요한 경우 각각 `--enable-kerberos`, `--enable-vnc` 를 추가한다.

`infra`의 `container` 생성 구현은 target host에 image가 없으면 pull하고, storage와 credential을 준비한 뒤 Docker container와 DB record를 생성한다.

8.2 기존 container에 새 image 적용

`docker restart` 는 기존 container가 참조하는 image를 그대로 다시 실행하므로 새로 push한 image를 적용하지 않는다. 공유 홈을 보존한 상태에서 새 날짜 tag를 지정해 container를 교체해야 한다.

교체 전에는 기존 container의 사용자, server, GPU, port, 만료일, VNC·Kerberos option을 기록하고 새 생성 계획과 비교한다. DB와 port record가 함께 관리되므로 운영 container를 `docker rm` 만으로 직접 교체하지 않는다. 새 container에서 SSH, Jupyter, 선택적인 VNC, 홈 쓰기와 GPU 인식을 확인한 뒤 이전 container를 정리한다.

9. 배포 확인과 rollback

배포가 끝나면 다음을 확인한다.

- registry와 target host가 동일한 날짜 tag를 사용하는가?
- 시작 log의 image 버전, CUDA/TensorFlow와 최소 driver가 manifest와 일치하는가?
- TensorFlow가 할당된 GPU를 인식하는가?
- 사용자 UID/GID와 primary·supplemental group이 공유 스토리지 권한과 일치하는가?
- 공유 홈에서 읽기·쓰기, SSH와 Jupyter가 동작하는가?
- Kerberos 사용 시 ccache가 보이고 keytab은 container에 없는가?

문제가 발견되면 `stable` 이나 `latest` 의 과거 의미를 추측하지 않고, 마지막으로 검증한 날짜 tag를 지정하여 container를 다시 생성한다. 문제가 있는 날짜 tag는 덮어쓰지 않고 manifest의 `support` 와 alias를 수정하여 신규 사용을 막은 뒤 원인을 수정한 새 날짜 tag를 배포한다.

10. 운영 안전 수칙

- 실제 사용자 password를 image layer나 manifest에 넣지 않는다.
- keytab을 image 또는 container mount로 제공하지 않는다.
- source 변경, local image, registry image와 실행 중인 container를 구분해 진단한다.
- 실제 push 전에는 `--dry-run`의 전체 tag 목록을 확인한다.
- `experimental` 이미지 버전을 `stable`로 승격하기 전에 대상 GPU test를 수행한다.
- 기존 공유 홈을 사용하는 test는 파일과 권한을 변경할 수 있으므로 별도 test 사용자와 경로에서 수행한다.